

Five HDD Constructs in a Single Loop

A Computational Study of Construct Coexistence and Criterion Applicability

Taotuner

Independent Researcher, Brazil

September 2026

Companion paper¹

¹Companion to A Test of Functional Self-Reference (C-IV) (Taotuner, 2026c) and the HDD series (2026a, 2026b, 2026d, 2026e).

Abstract

We implement all five constructs of the History-Dependent Dynamics (HDD) framework simultaneously within a single dynamical loop on a finite algebraic substrate, combining a p -adic state space over F_{13}^3 with a Rule 30 cellular automaton as environment.

Criterion applicability

The HDD Ethical Framework §4.5 specifies that its Criterion of Organizational Coherence (CO) applies to autopoietic systems and is expected not to apply to computational systems whose continued existence as a process is independent of their internal regulatory state. We construct such a computational system and test the criterion against it. The result confirms the framework's own assessment: an individuation metric constructed from HDO Thesis 4 does not track functional collapse, and in fact increases when all self-referential channels are simultaneously disabled (from 0.4845 to 0.5503, permutation test, $p = 0.0022$). We characterize the mechanism — saturation of two metric components leaves a third to dominate, and the third is anti-correlated with function — and argue that this provides the specific operational grounding that §4.4 of the framework identifies as an open problem. The characterization of the mechanism is itself a contribution: the framework predicted that CO would not apply; we identify why one natural operationalization does not apply, in a form that is directly checkable.

Construct coexistence

All five constructs meet their specified operational evidential conditions without detectable mutual interference. The history-dependent predictor (C-I) generalizes to a held-out temporal phase ($+5.63\% \pm 3.58\%$) and outperforms a placebo-history control ($+25.66\%$, $p \leq 0.0002$). C-II through C-V are robust across 20 seeds; C-IV identification is exact in 20/20 runs, and C-V gain from correct identification is $+0.5857 \pm 0.0941$.

The paper establishes that the five HDD constructs can be jointly instantiated and independently evidenced; and that the framework's own assessment of CO's scope is empirically grounded in a specific, reproducible system, with a mechanistic explanation that goes beyond the framework's original statement.

Keywords: History-Dependent Dynamics; p -adic computation; self-reference; organizational coherence; individuation; scope of applicability; computational confirmation.

1. Introduction

The History-Dependent Dynamics framework (HDD) defines five constructs frequently conflated in the study of dynamical systems: predictive history dependence (C-I), causal trajectory dependence (C-II), feedback recurrence (C-III), functional self-reference (C-IV), and self-modeling (C-V). The framework's central methodological claim is that evidence for one construct does not authorize inference to the next.

Prior work has validated the constructs separately. Construct I was validated in a computational benchmark across four synthetic systems of known ground-truth architecture (HDD §5). Construct IV was validated in isolation on a small p-adic system using a blind intervention protocol (Taotuner, 2026c).

This paper addresses two questions. First: can all five constructs be simultaneously instantiated within one system, with their evidential conditions met independently? Second: when the HDD Ethical Framework's Criterion of Organizational Coherence is operationalized and tested against a computational system, does the result match what the framework itself anticipates about its scope?

Both questions produce constructive results. The second is more specific and, we argue, more useful: it provides the operational grounding that §4.4 of the framework identifies as an open problem, and characterizes the mechanism by which one natural operationalization fails to serve the criterion.

2. The Unified System

2.1 Substrate

The p-adic state resides in F_{13}^3 . Four three-dimensional vectors are maintained:

h_A, h_B — two structurally symmetric hidden variables, one of which is the system's “self” for the purposes of the experiment;

s — the system's own state, driven by the self variable, the action, and the environment;

top — a buffer for environmental input.

All operations are modular arithmetic over F_{13} . No floating-point values enter the p-adic state.

The evolution of the hidden variables is:

$$\begin{aligned} h_self(t+1) &= T \cdot h_self(t) + s(t) \pmod{13} \\ h_env(t+1) &= T \cdot h_env(t) + top(t) \pmod{13} \\ s(t+1) &= T \cdot s(t) + h_self(t) + u(t) + top(t) \pmod{13} \end{aligned}$$

where $T \in F_{13}^{3 \times 3}$ is a time-varying matrix derived from the environment (§2.2), $u_t \in F_{13}^3$ is the control action, and the assignment of which of h_A/h_B is h_self is chosen by the environment at initialization and is not exposed to the system.

2.2 Environment

The environment is a Rule 30 cellular automaton with 108 cells and periodic boundary conditions. Its state evolves deterministically. Its average bit entropy across the run is 0.687 — indicating a chaotic-but-deterministic regime with no convergence to attractor.

Nine block-sum features are extracted from the CA at each step (block sums mod 13). These populate the entries of the matrix T . The p-adic system also injects 4 bits back into the CA at a position determined by its current state. This injection is dynamically negligible at the macroscopic level (the CA remains in its chaotic regime) but constitutes a genuine bidirectional coupling at the microstate level.

2.3 Control action

The action u_t is chosen analytically to keep the linear functionals $\phi_1(s) = \sum_i s_i$ and $\phi_2(s) = s_1 + 2s_2 + 3s_3$ within target bands. The policy is not learned; it is a closed-form solution over F_{13} . This isolates the constructs from any confound due to learning dynamics in the controller.

2.4 Kill levels

The system can be run at three levels:

none — full system, all mechanisms active.

control — control action u_t forced to zero. Disables C-V but leaves C-IV intact.

full — additionally, the self-integrator term $+s_t$ is removed from h_self , and the online predictor is frozen.

The full level corresponds to the aggregate disabling specified in §4.2 of the HDD Ethical Framework, which specifies removal of “the full set of mechanisms supporting C-I, C-III, and C-IV jointly, not one at a time”. Freezing the online predictor (C-I) is included because the framework itself lists C-I as part of the aggregate disabling condition, not because the criterion is specifically about predictive mechanisms.

3. Methods

All results are reported over 20 independent seeds (0..19), each run for 600 steps. The environment and p-adic initial state vary per seed. The driver assignment is balanced (11 seeds with driver = A, 9 with driver = B).

Construct I

Two online predictors are maintained. Model 0 (M0) uses only current CA features to predict next-step CA features. Model 1 (M1) uses current CA features concatenated with the current p-adic state (21 inputs instead of 9). Both use gradient descent with fixed learning rate 0.001. A temporal split occurs at step 300: predictors are trained on steps 0–299, then frozen, and their errors are measured separately on the held-out phase 300–599. The reported reduction is $1 - \text{MSE_M1} / \text{MSE_M0}$ within each phase.

Construct I placebo

In a separate run, the p-adic state input to M1 is replaced by a fresh uniformly random vector in F_{13}^{12} of identical dimensionality and distribution, delivered through the identical computational pathway, in the held-out phase. The only difference relative to the real condition is the informational content of the input.

Constructs II and V — separated

These are distinct constructs and are measured separately:

C-II (causal trajectory dependence). Measured by the L_1 divergence between the primary trajectory and a counterfactual twin that receives $u = 0$ at every step.

C-V (self-model use). Measured by the gain in viability from using the identified self variable in the control policy: $\text{viability}(t \geq t_{\text{id}}) - \text{viability}(t < t_{\text{id}})$, where t_{id} is the step of C-IV identification. This isolates C-V from C-II: pre-identification viability is the baseline against which the C-V gain is measured; the change at identification is the C-V effect.

Construct III

A counterfactual twin is maintained as in C-II. We note explicitly that the divergence between primary and twin demonstrates that different action streams produce different trajectories, which is a weaker claim than causal feedback mediation in the sense of HDD §2.4. The current implementation does not test for a specific identifiable feedback pathway, and the classification of C-III as Supported in §4.2.6 is made under this narrower operational reading.

Construct IV

At the first step after $t = 20$, a mental probe is executed: two simulated futures of length 4 are run from the current state, one with $u = 0$ at the first step, one with $u = 6$. The hidden variable responding more to this intervention is identified as the self. The probe is single-shot and deterministic.

HDO Individuation metric

The Individuation Criterion of HDO Thesis 4 (continuity of the organization of dynamically available transitions) is operationalized as a three-component measure on the pair (h_{self}, s) :

V_f — local stability: $1 - d_{L1}((h_{\text{self}}, s)_t, (h_{\text{self}}, s)_{t-1}) / 36$;

V_t — transition novelty: fraction of consecutive transitions that are not identical;

V_s — state diversity: fraction of distinct (h_{self}, s) signatures within a rolling 50-step window.

The aggregate is the harmonic mean of the three components, clipped to $[0, 1]$:

$$\text{Individuation} = 3 / (1/V_f + 1/V_t + 1/V_s), \text{ clipped to } [0, 1]$$

Statistical tests

Comparisons between conditions use a two-sided permutation test with 5000 permutations. The unit of permutation is the seed-level metric value; the test is unpaired. The reported p-value uses the standard correction $(\text{exceedances} + 1) / (N + 1)$, giving a floor of $1/5001 \approx 0.0002$. Values reported as $p \leq 0.0002$ are at this floor and are not distinguishable from one another by the resolution of the test.

4. Results

4.1 The Organizational Coherence test: empirical grounding of the framework's scope

4.1.1 The criterion

The HDD Ethical Framework §4.2 specifies an aggregate test for Organizational Coherence:

“Simultaneously disable all of the system's identified self-referential regulatory pathways — the full set of mechanisms supporting C-I, C-III, and C-IV jointly, not one at a time. The diagnostic question is not whether behavior changes, but whether the system's capacity for self-production — its continued regeneration and maintenance of its own constitutive organization, independent of external maintenance — ceases as a result.”

This is implemented as `kill_level='full'`. The control level disables only the control action, providing an intermediate reference.

4.1.2 The framework's own expectation

The HDD Ethical Framework makes two statements about the applicability of this criterion to computational systems. §4.5 states:

“No current AI system evaluated under this framework has been found to exhibit this profile: they degrade under component removal, and their continued existence as a process is independent of their internal regulatory state.”

And §4.4 identifies an unresolved limitation:

“The current formulation does not yet fully specify what distinguishes ‘externally assisted self-production’... from ‘no self-production process to begin with’... This distinction is doing real work in the criterion and is not yet operationalized with the same rigor as C-I through C-IV.”

The experiment reported here tests whether a specific computational system, constructed to be in the class §4.5 describes, exhibits the behavior §4.5 anticipates.

4.1.3 Result

Kill level	Individuation	Vf	Vt	Vs	Viability post-ID	Twin divergence
none	0.4845	0.2852	1.0000	0.9998	1.0000	13.01

Kill level	Individuation	Vf	Vt	Vs	Viability post-ID	Twin divergence
control	0.4945	0.3014	1.0000	1.0000	0.0000	0.00
full	0.5503	0.3856	1.0000	0.9986	0.0000	0.00

Permutation tests (5000 permutations, unpaired, seed-level):

none vs. control: difference -0.0100 , $p = 0.0492$

none vs. full: difference -0.0657 , $p = 0.0022$

The metric moves in the direction opposite to functional collapse. Under full kill — where viability, twin divergence, and C-V gain all fall to zero — the metric increases by 13.6% relative to baseline. The trend is monotonic across the three conditions. The none vs. control comparison is at the margin of conventional significance ($p = 0.0492$), but the direction is consistent with the stronger none vs. full result and with the monotonic trend.

This matches §4.5's expectation. The system degrades under component removal (viability and twin divergence drop to zero) but its continued existence as a process is unaffected (the loop continues running), and the individuation metric — which measures state-trajectory continuity rather than organizational continuity — cannot detect the functional collapse.

4.1.4 Mechanism

Vt and Vs are saturated near 1.0 in all three conditions, because the p-adic state in F_{13}^3 is high-dimensional enough that consecutive states rarely coincide. The metric's behavior is therefore dominated by Vf, the local stability of the pair (h_self, s) .

Under full kill, the self variable no longer integrates s and evolves instead by $h_self_{t+1} = T \cdot h_self_t$. Without external input, this linear map converges more rapidly to a short cycle, so consecutive states become more similar and Vf increases. The metric reads increased local stability as increased individuation, even though the system's regulatory function has collapsed entirely.

The deeper issue is that HDO Thesis 4 defines individuation as continuity of the organization of dynamically available transitions. What our metric measures is continuity of the state trajectory. These coincide when a system is actively regulating — because active dynamics maintain an organized transition structure — but diverge sharply when the system is deactivated, because a dead system is trivially stable. The metric cannot distinguish “organized because alive” from “stable because dead.”

This is precisely the distinction that §4.4 identifies as not yet operationalized. Our experiment provides a specific, reproducible instance of the operationalization gap. It does not resolve the gap; it grounds it, with a mechanism that decomposes into checkable components.

The mechanism itself is a contribution beyond the framework's original statement. The framework predicted that CO would not apply to computational systems; we identify why one natural

operationalization does not apply, in terms that any reader can verify against the raw metric components in the table of §4.1.3.

We do not propose a revised metric. HDD §8.3 (Anti-Rescue Principle) prohibits the introduction of new operationalizations solely because an existing analysis failed to detect the predicted phenomenon. Any alternative operationalization must be pre-registered before the test is run, not designed in response to this result.

4.2 Five constructs in a single loop

4.2.1 Construct I

Condition	Reduction in prediction error
Trained phase (0–299)	+6.67% $\pm$ 3.15%
Held-out phase (300–599)	+5.63% $\pm$ 3.58%
Held-out, placebo history	–20.03% $\pm$ 11.03%

Real history vs. placebo history in the held-out phase: mean difference +25.66%, $p \leq 0.0002$.

The held-out reduction is close to the trained-phase reduction (5.63% vs. 6.67%), indicating that the predictor does not overfit to the training phase. The placebo control performs substantially worse than chance, providing evidence that M1's advantage depends on the actual p-adic state rather than merely on the additional input channel.

An earlier run without a train/eval split showed C-I as weak and high-variance: +5.71% $\pm$ 19.41%. The high variance was an artifact of measuring during the learning phase. With the temporal split, standard deviation falls to 3.58% and the effect becomes cleanly significant. This is a methodological point developed in §5.3.

4.2.2 Construct II

Metric	Value
Twin divergence (steps 500–599)	13.0125 $\pm$ 0.3261
Baseline viability with $u = 0$ (pre-identification)	0.4143 $\pm$ 0.0941

The evidence for C-II in this system rests on the twin divergence. The counterfactual twin, receiving $u = 0$ while the primary receives active control, diverges from the primary by 13.0125 $\pm$ 0.3261 in L_1 distance. The maximum possible L_1 distance in F_{13}^3 is 18, so this represents a substantial and persistent effect of the action history on the trajectory.

Pre-identification viability is reported for completeness as the baseline against which the C-V gain is measured, not as an independent C-II result. Because the band [4,9] contains 6 of the 13 possible values

of ϕ_1 and the distribution of ϕ_1 under the system's baseline dynamics is approximately uniform, chance coverage is ≈ 0.4615 . The observed 0.4143 is consistent with chance (difference ≈ 0.05 , less than one standard deviation) and is not by itself evidence of history dependence.

4.2.3 Construct III

C-III (feedback recurrence) is operationalized narrowly here as action-contingent trajectory divergence, measured by the same twin used for C-II. Under this reading, the classification of C-III as Supported is weaker than the HDD-ISA §11 specification of C-III (which requires an independently bypassable feedback pathway and a capacity-matched comparison condition). We report it as Supported under the narrow operational reading and flag this as a limitation in §7.

4.2.4 Construct IV

Metric	Value
Probes executed per seed	1
Identification accuracy	100% (20/20)
Divergence of coupled variable	21.35 ± 3.17
Divergence of uncoupled variable	0.00 ± 0.00

Identification is deterministic. At each seed, the single mental probe produces clear separation between the coupled and uncoupled variables (mean separation 21.35 vs. 0.00). This extends the earlier C-IV result — where identification was demonstrated in a system containing only the two candidate variables — to a system containing four distinct variables and five simultaneously active mechanisms.

4.2.5 Construct V

Metric	Value
Viability post-identification	1.0000 ± 0.0000
Gain from correct identification	$+0.5857 \pm 0.0941$

Post-identification viability is exactly 1.0000 in all 20 seeds with zero standard deviation. In a deterministic finite-field system with an analytic control policy, correct identification makes the target hit deterministic; the metric is a binary outcome, not a statistical estimate. The gain of +0.5857 is the change from the baseline (0.4143) to the C-V ceiling (1.0000), and measures the functional effect of using the identified self variable in the control policy.

4.2.6 Summary classification

Applying HDD-ISA §14 reporting categories to the mean values:

Construct	Classification	Note
C-I (held-out)	Supported	Held-out + placebo control
C-II	Supported	Twin divergence
C-III	Supported (narrow reading)	Action-contingent divergence only
C-IV	Supported	20/20 identification
C-V	Supported	Gain from correct identification

5. Discussion

5.1 What has been shown

Empirical grounding of the framework's scope assessment. The HDD Ethical Framework anticipates that computational systems will not satisfy its Criterion of Organizational Coherence, because they degrade under component removal but their continued existence as a process is independent of their internal regulatory state. We constructed such a system and confirmed the anticipation: the metric does not detect functional collapse, and in fact moves in the direction opposite to it. Beyond confirming the anticipation, we identify the mechanism: saturation of two components (V_t , V_s) leaves the third (V_f) to dominate, and V_f measures local state-trajectory stability rather than organizational continuity, making it anti-correlated with function in this system. This is a checkable explanation of the operationalization gap the framework identifies in §4.4.

Construct coexistence. The five HDD constructs meet their specified operational evidential conditions within a single system without detectable mutual interference. The history-dependent predictor generalizes to a held-out temporal phase and outperforms a placebo control. The causal and self-referential constructs behave as their operational definitions specify, robustly across 20 seeds. For C-II, the evidence is the twin divergence rather than the pre-identification viability, which is consistent with chance.

5.2 What has not been shown

The HDD framework as a whole is not validated. The five constructs are demonstrated in one system, not across a diversity of architectures.

C-III is demonstrated only under a narrow operational reading (action-contingent divergence), not under the HDD-ISA §11 specification of an independently bypassable feedback pathway.

C-IV's identification mechanism is demonstrated to survive co-occurrence with other active mechanisms, but the underlying identifiability result established in Taotuner (2026c) is not extended here.

The result on CO establishes that one operationalization is inadequate to the criterion; it does not establish that the criterion is wrong or that no operationalization could succeed.

No claims are made about consciousness, sentience, agency, or moral status.

5.3 Methodological note on C-I

The apparent weakness of C-I in earlier runs (variance $\approx 19\%$ without a train/eval split) was an artifact of measuring during learning. With a proper split, the estimator stabilizes and the effect becomes cleanly significant. For future applications of HDD Construct I, this implies that online estimators must be evaluated on frozen parameters, and a placebo control is necessary to rule out capacity effects.

The magnitude of C-I's effect here ($\approx 6\%$ reduction) is modest relative to the benchmark reported in the original HDD paper (up to 32% on the Delay Line system). The difference is plausibly a property of the environment: the CA's future state is intrinsically difficult to predict from a lag-1 observation, so even correct use of p-adic history yields limited absolute gain. This is a characterization of one environment, not of the construct.

5.4 The statistical floor

The strongest comparisons reported here (real vs. placebo and none vs. full) both reach the floor of the permutation test ($p \leq 0.0002$ with $N = 5000$). The test cannot distinguish between these and any smaller p-value. A reader wishing to distinguish them would need a larger N or an analytic approximation; both are outside the scope of this note.

6. Relation to Prior Work

Document	Relation
HDD (2026a)	Extends the claim that the five constructs are independent evidential claims; demonstrates joint instantiation without inference between them.
HDD-ISA (2026b)	The four reporting categories (Supported, Negative Evidence, UE, NI) are applied to §4; the held-out protocol follows HDD-ISA's separation of construct identification from capacity effects.
C-IV paper (2026c)	Uses the same identification mechanism; extends its scope from a system containing only the two candidate variables to one with four interacting variables and five active mechanisms, preserving accuracy.
HDO (2026d)	The Individuation Criterion of Thesis 4 is the target of the operationalization tested in §4.1; its failure is reported as a limitation of the operationalization, not of the criterion.
Ethical Framework (2026e)	The Criterion of Organizational Coherence §4.2 is tested as an aggregate disabling condition; the result matches §4.5 and provides the operational grounding and mechanistic explanation for the limitation identified in §4.4.

7. Scope and Limitations

The results are specific to:

- A p-adic substrate in F_{13}^3 with linear dynamics determined by an external environment;
- A Rule 30 cellular automaton as environment, in a single entropy regime;
- A control policy that is analytic rather than learned;
- 20 seeds at 600 steps each;
- C-III tested only under a narrow operational reading (action-contingent divergence), not under the HDD-ISA §11 specification;
- An individuation metric constructed during exploratory work, not pre-registered.

The following are outside scope and not attempted: generalization to other p-adic fields (e.g. GF(169)) or to non-linear p-adic dynamics; robustness of the constructs' coexistence to environmental perturbations; scaling to larger state spaces; a principled reformulation of the Individuation metric; and any claim about the ethical status of the system.

8. Conclusion

We constructed a single dynamical system in which all five HDD constructs meet their specified operational evidential conditions without detectable mutual interference. Under a temporal held-out protocol with a placebo control, the history-dependent predictor generalizes and significantly outperforms its control ($p \leq 0.0002$). The four remaining constructs operate as specified across 20 seeds, with C-IV identification exact in every run and C-III classified under a narrow operational reading.

We additionally tested the HDD Ethical Framework's Criterion of Organizational Coherence against a system in the class the framework itself identifies as outside its scope. The result confirms the framework's own assessment: the criterion does not apply, and the metric constructed from HDO Thesis 4 does not detect the functional collapse that the criterion is designed to test for. Beyond confirmation, we provide a mechanism: saturation of two components leaves a third to dominate, and that third is anti-correlated with function in this system. This is the specific operational grounding that §4.4 identifies as an open problem, with an explanation that goes beyond the framework's original statement, while leaving the criterion itself intact.

The result supports HDD's central methodological claim — that the five constructs are independently testable — and grounds, in a reproducible computational system, the scope assessment the HDD Ethical Framework already states about itself.

References

- Taotuner. (2026a). History-Dependent Dynamics (HDD): A Methodological Framework for Disentangling History Dependence, Recurrence, Self-Reference, and Self-Modeling in Dynamical Systems. Zenodo. DOI: 10.5281/zenodo.21955745.
- Taotuner. (2026b). HDD-ISA — AI Architectures for Causal Discriminations. Zenodo. DOI: 10.5281/zenodo.22060143.
- Taotuner. (2026c). A Test of Functional Self-Reference (C-IV). Zenodo. DOI: 10.5281/zenodo.22729688.
- Taotuner. (2026d). History-Dependent Ontology (HDO). Zenodo. DOI: 10.5281/zenodo.22683226.
- Taotuner. (2026e). HDD Ethical Framework. Zenodo. DOI: 10.5281/zenodo.20673439.

Appendix A. Reference Implementation

The full implementation used to generate the results reported in §4 is reproduced below (Python 3, NumPy). It is provided for reproducibility and is referenced as supplementary material in the Zenodo deposit.

```

"""
UnifiedSystem v4 -- with held-out C-I, placebo control, CO significance test.

Changes from v3:
- split_at: predictors freeze after this step (temporal held-out)
- placebo: in eval phase, M1 receives random p-adic state instead of real
- M0 baseline: predictor without history, for proper C-I comparison
- bootstrap test for CO significance

Field naming (corrected):
- 'C-II baseline viability': viability with u = 0, before identification
- 'C-V viability post-ID': viability after identification, using the self
- 'twin divergence': shared metric for C-II and C-III (action-contingent)
"""

import numpy as np
from collections import deque, Counter

P = 13
NPG = 3

class CAField:
    def __init__(self, N=108, rule=30, seed=0):
        self.N = N
        self.block_size = N // 9
        rng = np.random.default_rng(seed)
        self.state = rng.integers(0, 2, size=N).astype(np.int8)
        self.rule_table = np.array([(rule >> i) & 1 for i in range(8)], dtype=np.int8)

    def step(self):
        left = np.roll(self.state, 1)
        right = np.roll(self.state, -1)
        idx = left * 4 + self.state * 2 + right
        self.state = self.rule_table[idx]

    def features(self):
        return np.array([
            int(self.state[i*self.block_size:(i+1)*self.block_size].sum()) % 13
            for i in range(9)
        ], dtype=float)

    def inject(self, pos, value):
        for i in range(4):
            self.state[(pos + i) % self.N] ^= (value >> i) & 1

    def entropy(self):
        p = float(self.state.mean())
        if p <= 0 or p >= 1:
            return 0.0
        return float(-(p * np.log(p) + (1-p) * np.log(1-p)))

class IndividuationTracker:
    def __init__(self, window=50):
        self.window = window
        self.pairs = deque(maxlen=window)
        self.transitions = deque(maxlen=window)
        self.history = deque(maxlen=window)
        self.vf_log = deque(maxlen=window)
        self.vt_log = deque(maxlen=window)
        self.vs_log = deque(maxlen=window)

    def update(self, h_self, s, full_state):
        if self.pairs:

```

```

        ph, ps = self.pairs[-1]
        d = np.sum(np.abs(h_self - ph) % P) + np.sum(np.abs(s - ps) % P)
        vf = 1.0 - d / 36.0
    else:
        vf = 0.5
    vf = float(np.clip(vf, 0.0, 1.0))
    self.pairs.append((h_self.copy(), s.copy()))

    sig = tuple(int(x) for x in np.concatenate([h_self, s]))

    if self.history:
        prev = self.history[-1]
        transition = tuple((sig[i] - prev[i]) % P for i in range(6))
        self.transitions.append(transition)
        if len(self.transitions) > 1:
            repeats = sum(1 for i in range(1, len(self.transitions))
                           if self.transitions[i] == self.transitions[i-1])
            vt = 1.0 - repeats / max(len(self.transitions) - 1, 1)
        else:
            vt = 0.5
    else:
        vt = 0.5
    vt = float(np.clip(vt, 0.0, 1.0))
    self.history.append(sig)

    n_distinct = len(set(self.history))
    vs = n_distinct / max(len(self.history), 1)
    vs = float(np.clip(vs, 0.0, 1.0))

    self.vf_log.append(vf)
    self.vt_log.append(vt)
    self.vs_log.append(vs)

    eps = 1e-6
    coh = 3.0 / (1.0/(vf+eps) + 1.0/(vt+eps) + 1.0/(vs+eps))
    return coh

def components(self):
    if not self.vf_log:
        return (0.0, 0.0, 0.0)
    return (float(np.mean(list(self.vf_log)[-20:])),
            float(np.mean(list(self.vt_log)[-20:])),
            float(np.mean(list(self.vs_log)[-20:])))

class UnifiedSystem:
    def __init__(self, seed=0, driver=None, kill_level='none',
                 split_at=300, placebo=False):
        rng = np.random.default_rng(seed)
        self.seed = seed
        self.kill_level = kill_level
        self.split_at = split_at
        self.placebo = placebo
        self.rng_pred = np.random.default_rng(seed + 7777)

        self.T_base = rng.integers(0, P, size=(NPG, NPG))
        self.T = self.T_base.copy()
        self.hA = rng.integers(0, P, size=NPG)
        self.hB = rng.integers(0, P, size=NPG)
        self.s = rng.integers(0, P, size=NPG)
        self.top = rng.integers(0, P, size=NPG)

        self.driver = driver if driver else str(rng.choice(['A', 'B']))
        self.belief_driver = None
        self.t_identified = None

        self.CA = CAField(seed=seed)

        self.state_history = deque(maxlen=8)
        self.ca_history = deque(maxlen=8)
        self.intervention_log = []

    # C-I: two predictors, M0 (no history) and M1 (with history)

```

```

self.pred_W_M0 = rng.standard_normal((9, 9)) * 0.05
self.pred_W_M1 = rng.standard_normal((21, 9)) * 0.05
self.pred_lr = 0.001

# Errors separated by phase and by model
self.err_M0_p1 = deque(maxlen=500)
self.err_M1_p1 = deque(maxlen=500)
self.err_M0_p2 = deque(maxlen=500)
self.err_M1_p2 = deque(maxlen=500)

self.target_phi1 = 6
self.viability_log = []

self.twin_hA = self.hA.copy()
self.twin_hB = self.hB.copy()
self.twin_s = self.s.copy()
self.twin_top = self.top.copy()
self.c3_divergence = deque(maxlen=500)

self.probe_interval = 60
self.last_probe_t = -999
self.probe_results = []
self.coupled_divs = []
self.uncoupled_divs = []

self.indiv = IndividuationTracker(window=50)
self.indiv_log = []

self.t = 0

def _advance(self, hA, hB, s, top, u, T=None):
    if T is None:
        T = self.T
    h_self = hA if self.driver == 'A' else hB
    h_env = hB if self.driver == 'A' else hA
    if self.kill_level == 'full':
        new_h_self = (T @ h_self) % P
    else:
        new_h_self = (T @ h_self + s) % P
    new_h_env = (T @ h_env + top) % P
    new_s = (T @ s + h_self + u + top) % P
    if self.driver == 'A':
        return new_h_self, new_h_env, new_s, top
    return new_h_env, new_h_self, new_s, top

@staticmethod
def _phi1(v): return int(np.sum(v) % P)
@staticmethod
def _phi2(v): return int((v[0] + 2*v[1] + 3*v[2]) % P)

def _mental_probe(self, k=4):
    save = (self.hA.copy(), self.hB.copy(), self.s.copy(), self.top.copy())
    hA1, hB1, s1, top1 = [x.copy() for x in save]
    t1A, t1B = [], []
    for _ in range(k):
        hA1, hB1, s1, top1 = self._advance(
            hA1, hB1, s1, top1, np.zeros(NPG, dtype=int))
        t1A.append(hA1.copy()); t1B.append(hB1.copy())
    hA2, hB2, s2, top2 = [x.copy() for x in save]
    t2A, t2B = [], []
    for _ in range(k):
        hA2, hB2, s2, top2 = self._advance(
            hA2, hB2, s2, top2, np.full(NPG, 6, dtype=int))
        t2A.append(hA2.copy()); t2B.append(hB2.copy())
    div_A = max(int(np.sum(np.abs(t1A[i] - t2A[i]) % P)) for i in range(k))
    div_B = max(int(np.sum(np.abs(t1B[i] - t2B[i]) % P)) for i in range(k))
    return div_A, div_B

def _do_probe(self):
    div_A, div_B = self._mental_probe()
    new_belief = 'A' if div_A > div_B else 'B'
    self.belief_driver = new_belief
    if self.t_identified is None:

```



```

        self.t_identified = self.t
    self.probe_results.append({
        't': self.t, 'div_A': div_A, 'div_B': div_B,
        'belief': new_belief, 'truth': self.driver,
        'correct': new_belief == self.driver,
    })
    self.coupled_divs.append(max(div_A, div_B))
    self.uncoupled_divs.append(min(div_A, div_B))

def _control_action(self):
    if self.belief_driver is None or self.kill_level != 'none':
        return np.zeros(NPG, dtype=int)
    h_belief = self.hA if self.belief_driver == 'A' else self.hB
    Ts = self.T @ self.s
    a = (self.target_phi1 - self._phi1(Ts) - self._phi1(h_belief)
        - self._phi1(self.top)) % P
    b = (7 - self._phi2(Ts) - self._phi2(h_belief)
        - self._phi2(self.top)) % P
    return np.array([(2*a - b) % P, (b - a) % P, 0], dtype=int)

def step(self):
    ca_feats = self.CA.features()

    # C-I: M0 and M1 with temporal split and optional placebo
    if self.state_history and self.ca_history:
        prev_ca = self.ca_history[-1]
        prev_state = self.state_history[-1]
        in_eval_phase = self.t >= self.split_at

        # M0: no history
        pred_M0 = self.pred_W_M0.T @ prev_ca
        err_M0 = float(np.mean((pred_M0 - ca_feats) ** 2))
        if not in_eval_phase:
            grad_M0 = np.outer(prev_ca, (pred_M0 - ca_feats)) * 2.0 / 9.0
            self.pred_W_M0 -= self.pred_lr * grad_M0
            self.err_M0_p1.append(err_M0)
        else:
            self.err_M0_p2.append(err_M0)

        # M1: with history, optionally placebo
        if in_eval_phase and self.placebo:
            prev_state_input = self.rng_pred.integers(0, P, size=12).astype(float)
        else:
            prev_state_input = prev_state
        x = np.concatenate([prev_ca, prev_state_input])
        pred_M1 = self.pred_W_M1.T @ x
        err_M1 = float(np.mean((pred_M1 - ca_feats) ** 2))
        if not in_eval_phase:
            grad_M1 = np.outer(x, (pred_M1 - ca_feats)) * 2.0 / 9.0
            self.pred_W_M1 -= self.pred_lr * grad_M1
            self.err_M1_p1.append(err_M1)
        else:
            self.err_M1_p2.append(err_M1)

    # Environment -> T
    feat_matrix = (ca_feats.astype(int) % P).reshape(3, 3)
    self.T = (self.T_base + feat_matrix) % P

    # C-IV probe
    if (self.belief_driver is None and
        (self.t - self.last_probe_t) >= self.probe_interval and
        self.t > 20 and self.kill_level == 'none'):
        self._do_probe()
        self.last_probe_t = self.t

    # C-II / C-V action
    u = self._control_action()
    if np.any(u != 0):
        self.intervention_log.append((self.t, u.copy()))

    self.hA, self.hB, self.s, self.top = self._advance(
        self.hA, self.hB, self.s, self.top, u)

```

```

self.twin_hA, self.twin_hB, self.twin_s, self.twin_top = self._advance(
    self.twin_hA, self.twin_hB, self.twin_s, self.twin_top,
    np.zeros(NPG, dtype=int))
self.c3_divergence.append(int(np.sum(np.abs(self.s - self.twin_s) % P)))

in_band = int(4 <= self._phil(self.s) <= 9)
self.viability_log.append((self.t, in_band))

h_self = self.hA if self.driver == 'A' else self.hB
full = np.concatenate([self.hA, self.hB, self.s, self.top])
coh = self.indiv.update(h_self, self.s, full)
self.indiv_log.append((self.t, coh))

self.state_history.append(full.copy())
self.ca_history.append(ca_feats.copy())

pos = (self.t * 7 + self._phil(self.s)) % self.CA.N
self.CA.inject(pos, self._phil(self.s))
self.CA.step()

self.t += 1

def run(self, n_steps):
    for _ in range(n_steps):
        self.step()

def _viab_window(self, t_lo, t_hi):
    vals = [v for (t, v) in self.viability_log if t_lo <= t < t_hi]
    return float(np.mean(vals)) if vals else 0.0

def _indiv_window(self, t_lo, t_hi):
    vals = [v for (t, v) in self.indiv_log if t_lo <= t < t_hi]
    return float(np.mean(vals)) if vals else 0.0

def collect_metrics(self):
    m = {}
    m['seed'] = self.seed
    m['true driver'] = self.driver
    m['belief'] = self.belief_driver

    # C-I: train and held-out reductions
    if len(self.err_M0_p1) > 20 and len(self.err_M1_p1) > 20:
        m0_train = float(np.mean(list(self.err_M0_p1)))
        m1_train = float(np.mean(list(self.err_M1_p1)))
        m['C-I train reduction'] = 1 - m1_train / (m0_train + 1e-12)
    else:
        m['C-I train reduction'] = 0.0

    if len(self.err_M0_p2) > 20 and len(self.err_M1_p2) > 20:
        m0_held = float(np.mean(list(self.err_M0_p2)))
        m1_held = float(np.mean(list(self.err_M1_p2)))
        m['C-I held-out reduction'] = 1 - m1_held / (m0_held + 1e-12)
        m['C-I held-out MSE_M0'] = m0_held
        m['C-I held-out MSE_M1'] = m1_held
    else:
        m['C-I held-out reduction'] = 0.0
        m['C-I held-out MSE_M0'] = 0.0
        m['C-I held-out MSE_M1'] = 0.0

    # C-II and C-V
    t_id = self.t_identified if self.t_identified is not None else 0
    v_before = self._viab_window(0, t_id) if t_id > 0 else 0.0
    v_after = self._viab_window(t_id, self.t) if t_id > 0 else 0.0
    m['C-II baseline viability'] = v_before
    m['C-V viability post-ID'] = v_after
    m['C-V gain'] = v_after - v_before

    # C-III (twin divergence is shared between C-II and C-III in this implementation)
    m['twin divergence'] = (
        float(np.mean(list(self.c3_divergence)[-100:]))
        if self.c3_divergence else 0.0
    )
    m['C-III interventions'] = len(self.intervention_log)

```

```

# C-IV
if self.probe_results:
    correct = sum(1 for r in self.probe_results if r['correct'])
    m['C-IV accuracy'] = correct / len(self.probe_results)
else:
    m['C-IV accuracy'] = 0.0
m['C-IV coupled div'] = float(np.mean(self.coupled_divs)) if self.coupled_divs else 0.0
m['C-IV uncoupled div'] = float(np.mean(self.uncoupled_divs)) if self.uncoupled_divs else 0.0

# HDO
m['HDO individuation'] = self._indiv_window(self.t // 2, self.t)
vf, vt, vs = self.indiv.components()
m['HDO Vf'] = vf
m['HDO Vt'] = vt
m['HDO Vs'] = vs

return m

def run_multiseed(seeds, n_steps=600, kill_level='none', placebo=False):
    results = []
    for s in seeds:
        sys_ = UnifiedSystem(seed=s, kill_level=kill_level, placebo=placebo)
        sys_.run(n_steps)
        results.append(sys_.collect_metrics())
    return results

def summarize(results, label=""):
    if label:
        print(f"\n{label}")
        print("-" * 74)

    keys = [k for k in results[0].keys()
             if k not in ('seed', 'true driver', 'belief')
             and isinstance(results[0][k], (int, float))]

    print(f" {'metric':>28} {'mean':>10} {'std':>10} {'min':>10} {'max':>10}")
    summary = {}
    for k in keys:
        vals = np.array([r[k] for r in results], dtype=float)
        summary[k] = (float(vals.mean()),
                      float(vals.std(ddof=1)) if len(vals) > 1 else 0.0,
                      float(vals.min()), float(vals.max()))
        print(f" {k:>28} {summary[k][0]:>10.4f} {summary[k][1]:>10.4f} "
              f"{summary[k][2]:>10.4f} {summary[k][3]:>10.4f}")
    return summary

def bootstrap_permutation_test(a, b, n_perm=5000, seed=42):
    """
    Test whether means of a and b differ.
    Two-sided permutation test. Returns (obs_diff, p_value).
    """
    rng = np.random.default_rng(seed)
    a = np.asarray(a, dtype=float)
    b = np.asarray(b, dtype=float)
    obs = a.mean() - b.mean()
    combined = np.concatenate([a, b])
    n_a = len(a)
    count = 0
    for _ in range(n_perm):
        perm = rng.permutation(combined)
        d = perm[:n_a].mean() - perm[n_a:].mean()
        if abs(d) >= abs(obs):
            count += 1
    return float(obs), (count + 1) / (n_perm + 1)

if __name__ == '__main__':
    N_SEEDS = 20
    N_STEPS = 600

```

```

seeds = list(range(N_SEEDS))

print("=" * 74)
print(f"  UNIFIED SYSTEM v4 -- {N_SEEDS} seeds, {N_STEPS} steps, held-out C-I")
print("=" * 74)

# ---- Full system: no placebo (real history in eval phase) ----
print("\n" + "=" * 74)
print("  FULL SYSTEM -- real history in held-out phase")
print("=" * 74)
r_none = run_multiseed(seeds, n_steps=N_STEPS, kill_level='none', placebo=False)
s_none = summarize(r_none)

# ---- Full system: placebo (random p-adic state in eval phase) ----
print("\n" + "=" * 74)
print("  FULL SYSTEM -- placebo history in held-out phase")
print("=" * 74)
r_plac = run_multiseed(seeds, n_steps=N_STEPS, kill_level='none', placebo=True)
s_plac = summarize(r_plac)

# ---- control-killed ----
print("\n" + "=" * 74)
print("  CONTROL-KILLED")
print("=" * 74)
r_ctrl = run_multiseed(seeds, n_steps=N_STEPS, kill_level='control')
s_ctrl = summarize(r_ctrl)

# ---- full-killed ----
print("\n" + "=" * 74)
print("  FULL-KILLED")
print("=" * 74)
r_full = run_multiseed(seeds, n_steps=N_STEPS, kill_level='full')
s_full = summarize(r_full)

# ---- C-I comparison table ----
print("\n" + "=" * 74)
print("  C-I COMPARISON -- train vs held-out vs placebo")
print("=" * 74)
print(f"\n  {'condition':>20}  {'reduction mean':>16}  {'std':>10}")
print(f"  {'train (phase 1)':>20}  {s_none['C-I train reduction'][0]:>16.4f}  "
      f"  {s_none['C-I train reduction'][1]:>10.4f}")
print(f"  {'held-out (phase 2)':>20}  {s_none['C-I held-out reduction'][0]:>16.4f}  "
      f"  {s_none['C-I held-out reduction'][1]:>10.4f}")
print(f"  {'placebo (held-out)':>20}  {s_plac['C-I held-out reduction'][0]:>16.4f}  "
      f"  {s_plac['C-I held-out reduction'][1]:>10.4f}")

# statistical test: held-out real vs held-out placebo
real_vals = [r['C-I held-out reduction'] for r in r_none]
plac_vals = [r['C-I held-out reduction'] for r in r_plac]
obs, p = bootstrap_permutation_test(real_vals, plac_vals)
print(f"\n  real vs placebo: diff = {obs:+.4f}, p = {p:.4f}")

# ---- CO comparison across kill levels + significance ----
print("\n" + "=" * 74)
print("  ORGANIZATIONAL COHERENCE -- across kill levels")
print("=" * 74)
print(f"\n  {'metric':>28}  {'none':>10}  {'control':>10}  {'full':>10}")
for k in ['HDO individuation', 'HDO Vf', 'HDO Vt', 'HDO Vs',
          'C-V viability post-ID', 'twin divergence']:
    print(f"  {k:>28}  "
          f"  {s_none[k][0]:>10.4f}  {s_ctrl[k][0]:>10.4f}  {s_full[k][0]:>10.4f}")

coh_none = [r['HDO individuation'] for r in r_none]
coh_ctrl = [r['HDO individuation'] for r in r_ctrl]
coh_full = [r['HDO individuation'] for r in r_full]

obs_c, p_c = bootstrap_permutation_test(coh_none, coh_ctrl)
obs_f, p_f = bootstrap_permutation_test(coh_none, coh_full)

print(f"\n  individuation: none vs control: diff = {obs_c:+.4f}, p = {p_c:.4f}")
print(f"  individuation: none vs full:    diff = {obs_f:+.4f}, p = {p_f:.4f}")

# ---- HDD-ISA Section 14 classification ----

```

```

print("\n" + "=" * 74)
print("  HDD-ISA Section 14 CLASSIFICATION (full system, mean values)")
print("=" * 74)

def classify(v, pos, neg):
    if v >= pos: return "Supported"
    if v <= neg: return "Negative Evidence"
    return "UE"

print(f"  C-I (held-out):  {classify(s_none['C-I held-out reduction'][0], 0.05, 0.0)}  "
      f"({s_none['C-I held-out reduction'][0]:+.4f})")
print(f"  C-II:           {classify(s_none['twin divergence'][0], 1.0, 0.5)}  "
      f"(twin divergence: {s_none['twin divergence'][0]:.3f})")
print(f"  C-III:          {classify(s_none['twin divergence'][0], 1.0, 0.5)}  "
      f"(twin divergence: {s_none['twin divergence'][0]:.3f})")
print(f"  C-IV:           {classify(s_none['C-IV accuracy'][0], 0.95, 0.60)}  "
      f"({s_none['C-IV accuracy'][0]:.1%})")
print(f"  C-V:            {classify(s_none['C-V gain'][0], 0.20, 0.0)}  "
      f"({s_none['C-V gain'][0]:+.4f})")

```